I/O management

Matteo Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



1

Introduction

One of the main functions of computer systems is to interact and exchange information with the outside world.

This function is performed through the Input/Output (*I/O) subsystem*.

2

I/O sub-system

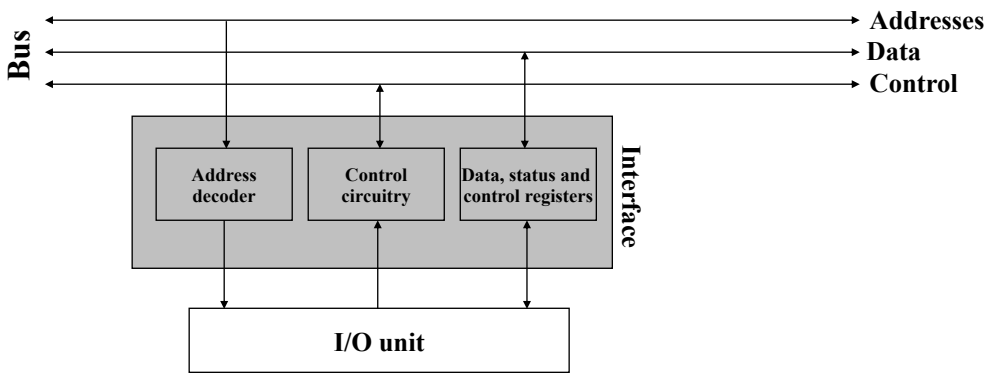
The I / O subsystem of a computer consists of:

- **Input / Output Devices (*peripherals*)**
- **Units for controlling these devices (*interfaces*)**
- **Software for their management.**

3

Adddressing I/O interfaces

Each I / O device is usually associated with an *interface* that connects it to the system bus.



4

Ports

Communication between the processor (or memory) and the device occurs through the interface registers (or *ports*).

These are accessible via the system bus, just like the memory cells, as each of them is associated with an address.

Their connection to the bus can take place in 2 modes:

- ***memory-mapped I / O***
- ***isolated I / O*** **(or I / O-mapped I / O).**

5

Memory-Mapped I/O

I / O interface registers are connected as normal memory cells.

The memory address space is therefore reduced.

You can access these registers using all the instructions and addressing methods that can be used to access the memory.

6

Isolated I/O

The addressing spaces for memory and I / O ports are separated, and are activated alternately by special signals (IO / M in 8086).

To access I / O ports, the programmer must use appropriate instructions (`IN` and `OUT` in 8086).

Synchronization

The operating speed of the CPU and the various peripheral devices can be very different.

The processor must then synchronize with I / O devices to perform the operations that interest them.

Synchronization can be done in various ways.

I/O management mechanism

They differ in the different levels of CPU engagement in I / O device management.

Possible solutions are:

- **Programmed I / O**
- **Interrupt**
- **Direct Memory Access (DMA).**

Programmed I/O

In this case, the management of the I / O devices is totally charged to the CPU.

For input operations, each data is first transferred from the port associated with the device to an internal CPU register, and then stored in memory (or vice versa).

Moving each data involves running at least one statement from the CPU.

Because the programmed I / O is based on the repetition of a status register test to see when the program can proceed further, it is also called *polling*.

Example

To transfer a data block to a parallel port using the programmed I / O, the CPU must execute a code snippet such as the following:

- `For each word in the block`
  1. `Check is the peripheral is ready by reading the status register`
  2. `If it is ready, write one data on the data register`
  3. `If not, go back to 1`

11

Features

The programmed I / O is normally used in smaller and less complex systems, as it has the following features:

+ **Rather cheap in terms of hardware**
- **Rather inefficient.**

12

Limitations of programmed I/O

Each device must depend on the CPU to be served.

Hence:

- **the efficiency (in terms of device usage) depends on the frequency with which the test is repeated**
- **all data must pass through the CPU, and there is no direct connection between device and memory**
- **the CPU dedicates a part of its time to performing trivial test and data transfer operations.**

Interrupt

It is based on an asynchronous signal that the device sends to the CPU when it needs a service.

In this way:

- **the time to get the attention of the CPU decreases**
- **the CPU does not waste time polling the device to test its status.**

Interrupt

I / O management via the interrupt mechanism can be used to increase CPU efficiency.

An asynchronous external input informs the microprocessor that a device requires to be served.

Upon arrival of a request to interrupt the CPU

- **Stops running the current program**
- **Moves to the execution of an *Interrupt Service Routine* (ISR).**

15

Interrupt Controller (IC)

The processor has (usually) a single signal on which to activate interrupt requests.

Peripherals that may require interrupt are usually more than one.

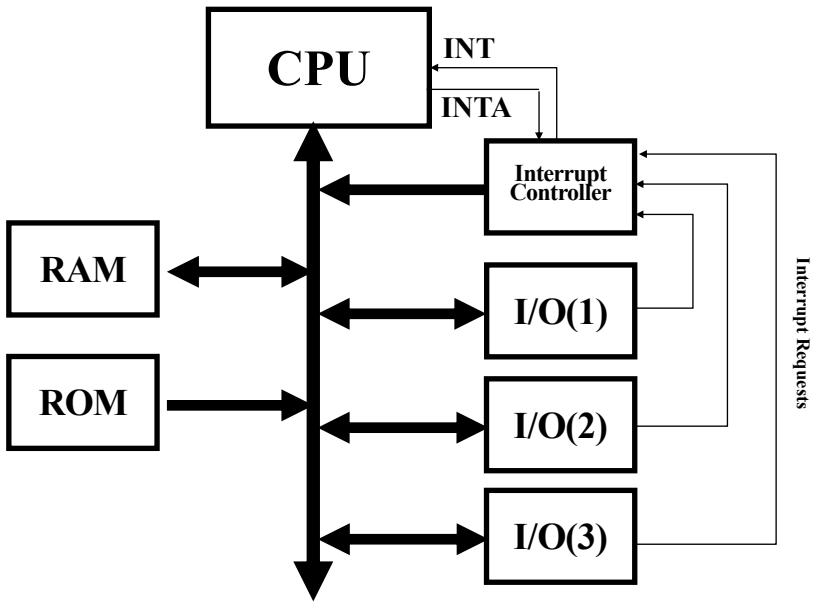
You need a module that

- **Receives all requests**
- **Drives the signal that goes to the processor.**

The *interrupt controller* works as a request manager and interfaces between peripheral interface devices and the CPU.

16

Interrupt



17

17

Interrupt Service Routines

They differ from normal procedures because their activation interrupts an unknown program.

Consequently, at the end of their execution, the program must be able to continue its activity as if it had not been interrupted.

Therefore, when an interrupt service is enabled, the following information are saved in the stack:

- the PC and the status word (via hardware)
- any register used by the procedure (via software).

At the end of the procedure you must

- Restore the register and status word value
- Return to the stopped program.

18

Interrupt latency

It is the time between the interrupt request and the start of the relative service procedure.

In some applications (such as those for real-time processing), it must be extremely small.

19

Interrupt disabling

In some cases, it is necessary to prevent an interrupt request from interrupting the processor, for example because it is performing an essential operation (often corresponding to another interrupt service procedure).

The following solution is usually adopted

- **there is a special enable bit for interrupts in the processor status register**
- **the bit is automatically set when the processor enters the service procedure**
- **the bit is reset at the end of the service procedure.**

20

Peripheral identification

When more than one peripheral requires an interrupt, a peripheral identification mechanism is required, whose service procedure must then be activated.

There are several solutions:

- **Multiple interrupt lines**
- **Polling**
- **Vectorized Interrupt.**

Multiple interrupt lines

In this case, the CPU has several input signals for receiving interrupt requests.

Each device is connected to a different signal.

This solution only works when the number of peripherals is very low (a few units).

Polling

In this case there is only one signal for interrupt requests.

When the CPU receives the request, it starts scanning the state words of all peripherals to locate the one that made the request.

Interrupt latency in this way is normally very high.

23

Vectorized interrupt

It is by far the most commonly adopted solution.

1. **When the processor is ready to serve the interrupt request, it sends an Acknowledge Interrupt signal**
2. **The peripheral interface (or the Interrupt Controller) puts its identification code on the data bus**
3. **The code is used by the processor to determine the address of the service procedure, using it as an index to access a vector named *Interrupt Vector Table* (IVT).**

24

Interrupt Vector Table

It is a table stored in the main memory (often starting at the address 000…00) that contains, for each type of interrupt, the address of its Interrupt Service Routine.

25

Peripheral priorities

In complex systems, it is often necessary to assign a priority to each peripheral, and to ensure that the service procedure of an interrupt is only interrupted by an interrupt request coming from a lower priority device.

In some cases, priorities assigned to peripherals can change over time.

The priority of the currently running procedure can be written in the status word.

In other cases, it is the interrupt controller that manages the priorities.

26

Simultaneous requests

In the case of simultaneous interrupt requests, the highest priority device must be served first .

If you use the mechanism of multiple interrupt lines, it is the processor that solves the problem.

In the case of polling, the processor first serves the peripheral that first appears in the sequence in which peripherals are polled.

In the case of the vectorized interrupt, there is almost always an Interrupt Controller who decides which is the first peripheral to be served, and provides the CPU with its ID code.

27

Interrupt sequence

1. **A peripheral activates the interrupt request to the IC**
2. **The IC activates the interrupt request to the CPU**
3. **The CPU activates the Interrupt Acknowledge signal**
4. **The IC sends to the CPU the code of the peripheral that made the request**
5. **The CPU saves the PC and the status register**
6. **The CPU accesses the IVT and extracts the ISR's address**
7. **The ISR's execution starts**
8. **At the end of the ISR**
    1. **the status registers is restored**
    2. **the interrupted program resumes execution.**

28

Exceptions

These events can interrupt the execution of the currently running program.

There are several types of exceptions:

- **I / O (or external) interrupts**
- **Error exceptions**
- **Debug exceptions**
- **Privilege exceptions.**

Each of these exception requests activates appropriate service procedures, whose address is contained in the IVT.

29

Error exceptions

They can be generated (among other things)

- **from memory control units, for example when a parity error occurs**
- **from the instruction decoding circuit, when the operating code of the fetched instruction does not match any of the legal instructions**
- **by arithmetic circuits, when an error condition occurs (for example, a division by 0).**

30

Debug exceptions

They are used by *debuggers*.

Many processors provide a *trace mode*; when activated, the processor triggers an exception after the execution of each statement.

This exception is used to handle the *single step* execution mode.

In addition, processors often have special instructions that trigger a desired exception.

These can be used to implement *breakpoints*.

31

Privilege exceptions

If the processor has multiple operating modes based on the priority of the running program (for example, *user* or *supervisor*), an exception is triggered when trying to

- **execute an unauthorized instruction from the current priority level, or**
- **access a memory area without having the rights.**

32

Interrupt usage

Operating Systems use heavily the interrupt mechanism:

- **the procedures that make up the Operating System (for example, device management) are activated**
  - **through software interrupts from the programs**
  - **through hardware interrupt requests from the peripherals themselves**
- **When activating interrupt procedures the CPU often passes to a special mode (*supervisor*).**

33

DMA

This is the preferred method when large amounts of data must be transferred from a peripheral device to memory (or vice versa).

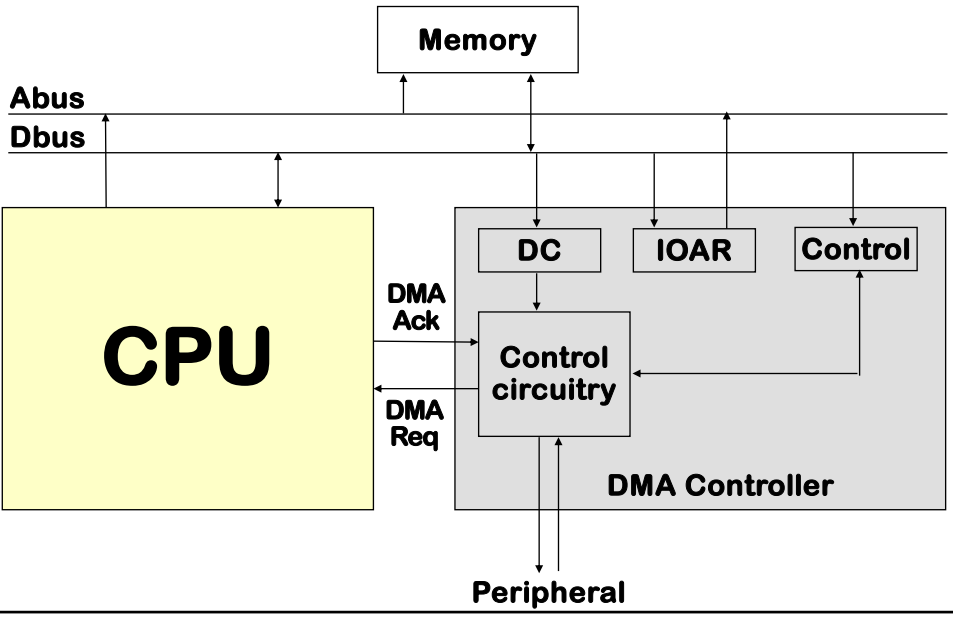
The transfer is performed by a dedicated module (*DMA Controller*).

The module must be able to act as a master bus, which means that it must be able to generate the addresses and control signals according to the bus protocol.

The DMA Controller must also be able to negotiate with the CPU the acquisition of the bus control and its release.

34

DMA controller



35

DMA channels

Each DMA controller can normally handle multiple peripherals (*channels*).

For each channel the DMA controller has

- request and acknowledge lines
- DC and IOAR registers
- control circuitry.

The DMA controller is also able to arbitrate between simultaneous DMA requests.

36

DMAC usage

The usage of the DMAC passes through 2 phases:

- ***Programming*:**
  the CPU writes the address of the memory area and the number of words to be transferred to the IOAR and DC registers, respectively; the CPU also informs the DMA controller of the transfer direction (memory → device or vice versa) and defines the transfer mode
- ***Usage*:**
  the DMAC performs the transfer.

DMA transfer (I)

The transfer of a block via DMA (after programming the DMAC) is performed in various steps:

- **the DMA Controller receives a transfer request from a peripheral device**
- **the DMA Controller sends a DMA Request signal to the CPU**
- **when the CPU receives a DMA Request signal detection point, it releases the bus and activates the DMA Acknowledge signal**

DMA transfer (II)

- **the DMA Controller starts the transfer; after the transfer of each word, IOAR and DC are updated**
- **the DMA Controller may temporarily suspend the transfer (for example, because the device has no data to transfer) by disabling DMA Request; the CPU disables DMA Acknowledge, and takes the bus control again**
- **when DC reaches zero, the transfer ends**
- **the DMA Controller sends a CPU Interrupt.**

39

DMA: operating modes

Data transfer via DMA may happen in different modes:

- ***burst transfer***
- ***cycle stealing***
- ***transparent DMA*.**

40

Burst transfer

In this operating mode, once the bus control has been acquired, the DMA Controller keeps it for as long as necessary to transfer a data block.

Thereby

- **the transfer takes place at maximum speed**

but

- **the CPU is locked for the duration of the transfer.**

Block transfer is important for peripherals such as magnetic disks, where block transfer can not be interrupted.

41

Cycle Stealing

The DMA Controller transfers the data into small blocks (for example, 1 word), occupying the bus for limited periods of time.

At the end of each block, the DMAC releases the bus and immediately afterwards activates the bus request.

Thereby:

- **the transfer rate is lower**

but

- **the CPU is not blocked for too long periods.**

42

Transparent DMA

The DMA Controller can detect when the CPU does not use the bus (for example, through specific status signals produced by the CPU), and only during those periods it performs the data transfer.

In this way the CPU is not virtually slowed down by the DMA transfer.

Bus arbitration

The DMA Controller is one of the devices that can become the master of the bus.

A device can be the master of the bus if it is able to decide what to do on the bus (read, write, etc.).

If there are several DMA Controllers, the system must include the circuitry for bus arbitration.